INTERNSHIP REPORT

Submitted

For the Partial Fulfilment of the Requirement for the Degree of Bachelor of _____ with _____ (Major and Minor Subjects) under **Magadh University**

Submitted to

The Department of _____, _____ **(College Name)**

[Paste College Logo here]

Submitted by

Name : _____

Semester : _____

Programme FYUGP : _____

University's Registration No. : _____

College Roll No. : _____

Year : _____

RK NEXORA
CAMPUS TO CAREER
A Unit of RKNexora Private Limited.

INTERNSHIP REPORT

Topic - Software Development

Submitted to the Head of Department, _____ (College Name), in partial fulfilment of the requirements for the Degree of _____, Magadh University

Name of Student : _____

Programme : _____

University Registration No. : _____

College Roll No. : _____

Internship Registration No. : _____

Name of the IPO / Agency : RKNEXORA PRIVATE LIMITED

Internship Period : _____ to _____

Semester : _____

Year / Session : _____



Signature of Student

Seal & Signature of IPO

ACKNOWLEDGEMENT

I express my sincere gratitude to RK NEXORA, RKNEXORA PRIVATE LIMITED, _____ (College Name), my faculty mentors and the internship supervisor, _____, for their guidance and support. The structured activities, learning resources and regular feedback helped me complete this internship in Software Development successfully.

I am thankful to the Head of the Department and the college mentor for their encouragement and for guiding me in preparing this report. I am also grateful to the management of RKNEXORA PRIVATE LIMITED for providing a professional learning environment and the opportunity to learn and apply my knowledge in a practical setting. The concept sessions, hands-on tasks with sample data and case studies, and the supervisor's patient and detailed feedback helped me understand how software is planned, designed, coded, tested and released.

Finally, I thank my family and peers for their constant encouragement throughout the programme.

_____

Name: _____

University Registration No. - _____

INTERNSHIP COMPLETION CERTIFICATE

The Internship Completion Certificate issued by RKNEXORA PRIVATE LIMITED for the internship in Software Development is to be attached on this page.

[Attach the original / printed copy of the QR-verified Internship Completion Certificate here]

Certificate No. _____

CONTENTS

Acknowledgement..... 3

Internship Completion Certificate..... 4

Preface..... 6

Executive Summary..... 7

About the Agency..... 8

Plan and Execution..... 9

Learning Outcome..... 22

Conclusion..... 24

Attendance Sheet..... 26

PREFACE

Internships are an important part of the undergraduate programme, as they allow students to see how classroom learning is applied in real organisations. The Four-Year Undergraduate Programme (FYUGP), introduced in line with the National Education Policy 2020, gives special importance to experiential learning, skill development and employability. This report has been prepared as part of the internship requirement of Magadh University and the Higher Education guidelines applicable in Bihar.

The report describes the internship undertaken in Software Development at RKNEXORA PRIVATE LIMITED, the plan followed, the work carried out, and the learning gained. Software now runs almost every part of daily life - the mobile payment that a shopkeeper receives, the railway ticket that a student books, the attendance system of a college. Behind each of them is a team that has understood a need, designed a solution, written and tested code and released it for use. Software development is therefore not only about writing code; it is a disciplined process that includes requirements, design, version control, testing, debugging and deployment. This report describes how the main stages of that process were studied and how a small stock management application was developed, tested and exposed through an API for a fictitious shop, using dummy data.

The report has been arranged in the order suggested by the University, beginning with the cover page and acknowledgement and ending with the conclusion and attendance sheet. It is hoped that it gives a clear and honest account of the internship experience. All examples in this report use dummy data and fictitious names; no real personal or business information has been handled.

EXECUTIVE SUMMARY

This report documents the internship completed in Software Development at RKNEXORA PRIVATE LIMITED (RK NEXORA - 'Campus to Career'). The programme required a minimum of 120 hours, completed over about four to five weeks (28 working days) with daily sessions of four to five hours, and was supported by an attendance log, supervisor guidance and review.

Background. Software development is the process of conceiving, specifying, designing, programming, testing, releasing and maintaining software. The process is described by the Software Development Life Cycle (SDLC), which has phases such as planning, requirement analysis, design, coding, testing, deployment and maintenance, and is organised through models such as waterfall and agile. Supporting practices include version control with Git, clean coding habits, database design, debugging, automated testing and the use of APIs.

Objectives. The internship aimed to (i) understand the SDLC and its models, (ii) collect and document requirements, (iii) prepare a simple system design, (iv) use Git for version control, (v) follow good coding practices, (vi) design a database, (vii) find and fix bugs, (viii) write and run tests, (ix) build and use a simple API, and (x) understand how an application is deployed.

Work carried out. Each topic had a concept session followed by hands-on work. The student wrote the requirements for a stock management application for a fictitious shop, designed its modules and database, built it in Python with SQLite, kept the code under Git with a feature branch, wrote automated tests, found and fixed a bug, exposed the application as a JSON API and tested it with curl. The later part of the programme was devoted to a case study, a mock practical exercise, supervisor review and preparation of this report.

Project. The work was tied together in a project titled **Software Application Development Project**. The project developed a small stock management application (store.py with SQLite, app.py as a JSON API) for a fictitious shop, with six automated tests, a Git history and a bug found and fixed through testing.

Method. A blended method was followed - learn, practise, document and review - supported by a daily attendance log with day-wise remarks and regular feedback from the supervisor.

Learning gained. The student gained a clear understanding of the SDLC, requirements and design, practical skill in Git, clean coding, database design, debugging, unit testing, APIs and deployment basics, and the habit of working in small, tested steps.

Key observations. Most mistakes in software come from unclear requirements rather than from wrong syntax; small commits and tests make change safe; a bug is cheaper to fix the earlier it is found; and readable code is as important as working code.

Conclusion. The internship met its objectives and contributed to the student's academic understanding, employability and confidence. The report follows the University's suggestive format: cover page, acknowledgement, certificate, contents, preface, executive summary, about the agency, plan and execution, learning outcome, conclusion and attendance sheet.

ABOUT THE AGENCY

RK NEXORA, a unit of RKNEXORA PRIVATE LIMITED, provides structured internship and skill-development programmes under the theme 'Campus to Career'. The organization focuses on academic-industry alignment, practical exposure, assessment, documentation and employability-oriented learning.

Particulars	Details
Name of Agency	RKNEXORA PRIVATE LIMITED
Programme / Unit	RK NEXORA - Campus to Career
Registered Office	C/O Mukesh Kumar, Khiri, PS Atri, Jethiyan, Atri, Gaya - 805236, Bihar, India
CIN	U85499BR2026PTC086757
Email	rknexorapvtltd@gmail.com
Website	www.rknexora.org
Internship Topic	Software Development

1. Vision

To support students in developing practical, professional and industry-relevant capabilities.

2. Mission

To deliver structured learning experiences that connect academic knowledge with workplace expectations.

3. Activities of the Agency

1. Structured internship programmes aligned with university and higher-education requirements.
2. Skill-development learning modules, practical activities and assignments.
3. Attendance tracking, supervision, assessment and documentation.
4. Employability-oriented guidance connecting campus learning to career readiness.

4. Programme Structure

The internship programme is organised around a fixed number of learning hours, completed over a defined period. Each intern is registered on the internship portal with a unique registration number, is assigned a supervisor, follows a day-wise plan and maintains an attendance log. Learning is organised into three phases: **Orientation** (Days 1-2), **Learning and Practice** (Days 3-22, ten sub-topics of two days each: one concept day and one hands-on day) and **Project and Review** (Days 23-28), followed by a concluding review.

5. Role of the Supervisor

The internship supervisor assigned by the agency guided the student through the programme, explained concepts, assigned practice tasks, reviewed the daily log and gave feedback. The supervisor also verified attendance and reviewed the student's work at the end of the programme.

PLAN AND EXECUTION

1. Objectives

The internship was planned around the following objectives, each linked to a part of the programme:

1. Understand the SDLC phases and the waterfall and agile models (Week 1).
2. Collect, document and validate requirements (Week 1).
3. Prepare a simple system and database design (Week 1-2).
4. Use Git: commits, branches and merges (Week 2).
5. Follow coding practices and debug programs (Week 2-3).
6. Write unit tests and report bugs (Week 3).
7. Build and use an API and understand deployment (Weeks 3-4).
8. Complete the project, report and review (Weeks 4-5).

2. Methodology

The programme followed a blended method made up of four connected steps, repeated for each topic:

Learn. Each topic began with a concept session in which the supervisor explained the idea, its main terms and how it is used in real organisations. Notes were prepared in the student's own words.

Practise. The concepts were then applied in guided hands-on tasks - writing a requirements document, drawing a data-flow and ER diagram, using Git branches and merges, writing clean Python, designing SQL tables with keys and constraints, tracing a bug, writing unit tests, building a JSON API and calling it with curl. Only sample data and fictitious examples were used, so that no real personal or business information was handled.

Document. Every day's work was recorded in the internship attendance log with a short remark, and sheets, files and screenshots were saved so that the work could be reviewed.

Review. The supervisor reviewed the work, pointed out errors and suggested improvements. Review days, a case study and a mock practical exercise helped to consolidate the learning.

3. Understanding Software Development - Concepts Studied

3.1 The software development life cycle

The SDLC is the series of phases through which software passes from an idea to a retired product. In planning, the need, cost, time and risks are studied. In requirement analysis, what the software must do is written down. In design, the structure of the system and the database are prepared. In coding, the design is turned into programs. In testing, the software is checked against the requirements. In deployment, it is released to users, and in maintenance it is corrected and improved over the years. The phases are shown below.

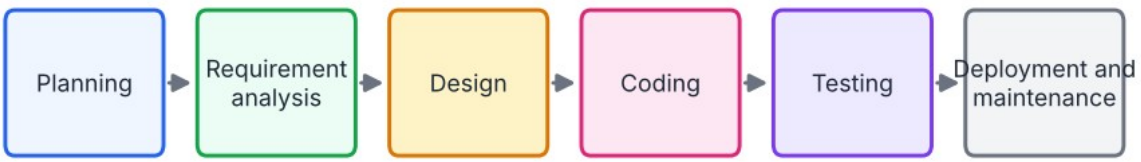


Figure 1: Phases of the software development life cycle

The waterfall model completes each phase before the next begins; it suits projects whose requirements are fixed and well understood, but a mistake found late is costly. The agile model works in short cycles, called sprints, of one to two weeks, at the end of which a working part of the software is shown to the customer and feedback is taken. It suits projects where requirements change. Agile work is organised around a product backlog (the list of wanted features), sprint planning, a daily short meeting, a review and a retrospective. The practice assignment used agile ideas: the stock application was built feature by feature, each tested before the next was started.



Figure 2: The agile cycle

3.2 Requirement analysis

A requirement is a statement of what the software must do or the qualities it must have. Functional requirements describe behaviour (for example, 'the shop-keeper can record a sale and the stock reduces'), and non-functional requirements describe qualities such as speed, security, usability and reliability. Requirements are collected through interviews, observation, questionnaires and study of existing documents, and are then written clearly, checked for conflicts and approved. A good requirement is clear, testable, complete and traceable to a need. Requirements for the dummy stock application are shown below.

ID	Requirement	Type	How it is tested
R1	The system shall store items with a name, price and quantity.	Functional	Add an item and read it back
R2	The system shall reduce the stock when a sale is recorded and return the amount.	Functional	Sell 3 bags of rice at Rs. 320; amount 960, stock 37
R3	The system shall refuse a sale larger than the stock.	Functional	Try to sell 9 when 8 are in stock
R4	The system shall list items whose stock is at or below a limit.	Functional	Low-stock list shows sugar
R5	The system shall refuse a negative price or quantity.	Functional / data quality	Tests with -1 and -5
R6	The system shall respond to an API call in under one second.	Non-functional	Timing of a call

A user story is a short way of writing a requirement from the user's side: 'As a shopkeeper, I want to see which items are running low, so that I can reorder them in time.' Acceptance criteria are added to each story so that everyone knows when it is complete.

3.3 System design basics

Design converts the requirements into a plan for construction. Architecture design decides the major parts and how they communicate, and detailed design decides the structure of each part. A layered design, in which a user-facing layer calls a logic layer, which calls a data layer, makes the software easier to change and test. In the stock application, app.py handles HTTP and JSON, store.py holds the rules of the shop (for example, that stock cannot go below zero) and SQLite stores the data. Diagrams used in design include the data flow diagram, the flowchart, the ER diagram and, for object-oriented design, the class diagram.

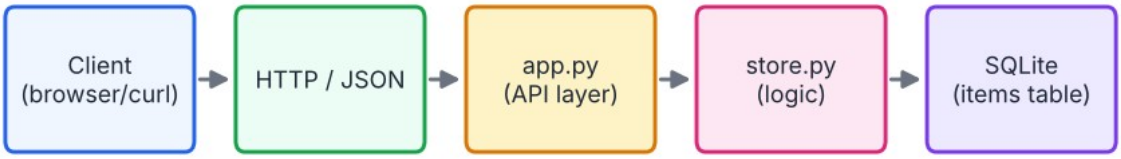


Figure 3: Architecture of the stock management application

Good design follows a few simple principles: each module has a single responsibility, modules depend on each other as little as possible (low coupling), the contents of a module belong together (high cohesion), and the same logic is not written in two places (DRY - do not repeat yourself).

3.4 Version control with Git

Version control records the history of the files of a project so that any earlier version can be recovered, changes can be compared and many people can work together. Git is the most widely used system. A repository holds the files and their history; a commit is a saved snapshot with a message; a branch is a separate line of work; a merge joins two branches; and a remote repository on a service such as GitHub lets a team share the work. The basic commands are git init, git add, git commit, git status, git log, git branch, git checkout (or switch), git merge, git pull and git push. Good practice is to make small commits with clear messages, to work on features in branches and to review changes before they are merged. The history of the project is shown below.

git history

```
$ git log --oneline --graph --all
*   b6cd2a3 Merge feature/low-stock-api
|\
| * faf6dcc Fix: reject zero/negative sale quantity (stock grew on negative n)
|/
* d292acc Add JSON API (GET /items, POST /sell)
* 0718bc8 Add unit tests for Store
* ef1af66 Add Store class with SQLite table
```

Figure 4: Git history of the project (feature branch merged into main)

The history shows three commits on the main branch (the Store class, the tests and the API), a feature branch in which the bug fix was made, and a merge commit that brought the fix into main. A merge conflict happens when two branches change the same lines; Git then marks the conflict and a person decides which version to keep.

3.5 Coding practices

Code is read many more times than it is written, so it should be easy to read. The practices followed were meaningful names (low_stock, not ls), short functions that do one thing, consistent indentation and formatting (in Python, the PEP 8 guide), comments that explain why rather than what, documentation strings for modules and functions, no repeated code, handling of errors with clear messages, and keeping secrets such as passwords out of the code. A code review by another person before merging finds mistakes and spreads knowledge. The main module of the project is shown below.

```
"""store.py - a small stock manager backed by SQLite (dummy shop data)."""
import sqlite3

class OutOfStock(Exception):
    pass

class Store:
    def __init__(self, path=":memory:"):
        self.db = sqlite3.connect(path)
        self.db.execute("""CREATE TABLE IF NOT EXISTS items(
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL UNIQUE,
            price REAL NOT NULL CHECK(price >= 0),
            qty INTEGER NOT NULL CHECK(qty >= 0))""")

    def add_item(self, name, price, qty):
        cur = self.db.execute("INSERT INTO items(name,price,qty) VALUES(?,?,?)",
                              (name, price, qty))
        self.db.commit()
        return cur.lastrowid

    def sell(self, name, n):
        if n <= 0:
            raise ValueError("quantity must be positive")
        row = self.db.execute("SELECT qty,price FROM items WHERE name=?",
(name,)).fetchone()
        if row is None:
            raise KeyError(name)
        qty, price = row
        if n > qty:
            raise OutOfStock(f"{name}: wanted {n}, have {qty}")
        self.db.execute("UPDATE items SET qty=qty-? WHERE name=?", (n, name))
        self.db.commit()
        return n * price

    def low_stock(self, limit=10):
        q = "SELECT name,qty FROM items WHERE qty<=? ORDER BY qty"
        return self.db.execute(q, (limit,)).fetchall()

    def all_items(self):
        return self.db.execute("SELECT id,name,price,qty FROM items ORDER BY
id").fetchall()
```

In this code the database itself rejects a negative price or quantity through CHECK constraints, a custom exception OutOfStock tells the caller exactly what went wrong, and parameterised queries (the question marks) prevent SQL injection, an attack in which a user types SQL into an input field.

3.6 Database design

Most applications store data in a relational database made of tables. Each table has a primary key that identifies a row, and tables are linked by foreign keys. Design begins by listing the entities (customers, items, sales), their attributes and the relations between them. Normalisation removes repeated data: in the first normal form each field holds a single value, in the second every non-key field depends on the whole key, and in the third no non-key field depends on another non-key field. For example, instead of repeating the name and phone of a customer on every sale row, the customer is stored once and the sale holds only the customer_id. The ER diagram below shows a design for the full shop; the project implemented the items table.

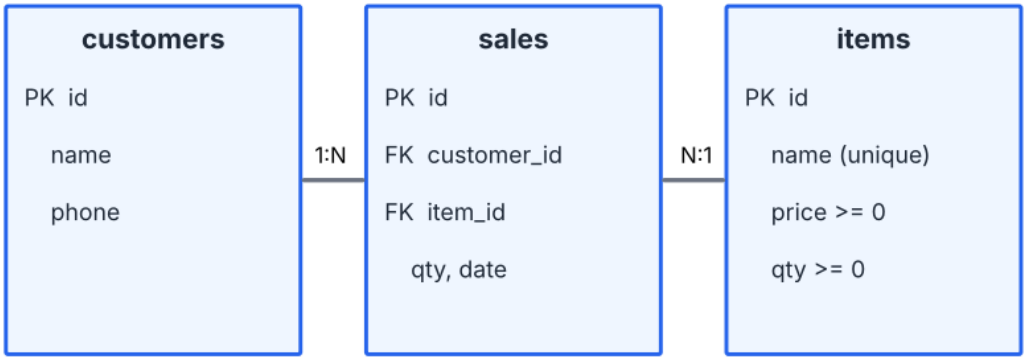


Figure 5: ER diagram of the shop database

```
CREATE TABLE items(
    id    INTEGER PRIMARY KEY AUTOINCREMENT,
    name  TEXT    NOT NULL UNIQUE,
    price REAL    NOT NULL CHECK(price >= 0),
    qty   INTEGER NOT NULL CHECK(qty >= 0)
);

-- items running low
SELECT name, qty FROM items WHERE qty <= 10 ORDER BY qty;

-- value of the stock
SELECT SUM(price * qty) AS stock_value FROM items;
```

The stock value query on the dummy data (rice 40 x 320, sugar 8 x 45, tea 25 x 110) gives 12,800 + 360 + 2,750 = Rs. 15,910.

3.7 Debugging techniques

A bug is a flaw that makes software behave differently from what is intended. Debugging is the systematic search for its cause. The method is to reproduce the problem reliably, narrow down where it occurs (by reading logs, printing values or using a debugger with breakpoints), form a guess about the cause, test the guess, fix the cause and not just the symptom, and add a test so that the bug does not return. Common causes are wrong conditions, off-by-one errors, unexpected input, wrong assumptions about data and changes made elsewhere. During the practice sessions, a bug was found in the first version of the sell function: a negative quantity was accepted and increased the stock.

bug found

```
$ python bug_demo.py
Before: [(1, 'Tea', 110.0, 25)]
sell('Tea', -5) returned -550.0
After : [(1, 'Tea', 110.0, 30)]  <-- stock grew: BUG
```

Figure 6: The bug - a negative sale quantity increased the stock

The cause was that the function checked only whether the quantity was more than the stock, and never whether it was positive. The fix was to reject any quantity of zero or less before the database is touched, and a new test was added. The fix was made on a separate branch and merged after all tests passed, as shown in Figure 4.

3.8 Software testing

Testing checks that the software meets its requirements and finds defects before users do. The levels are unit testing (one function or class), integration testing (parts together), system testing (the whole) and acceptance testing (by the customer). Methods are black-box testing (from outside, without looking at the code) and white-box testing (using knowledge of the code). A test case has an input, an expected result and the actual result. A unit test framework such as unittest in Python runs the tests automatically, so that they can be repeated after every change; this repeating is called regression testing. A bug report should have a title, the steps to reproduce, the expected and actual result, the severity and the version. The tests written for the project are shown below.

unit tests

```
$ python -m unittest -v test_store
test_cannot_sell_more_than_stock (test_cannot_sell_more_than_stock) ... ok
test_low_stock_list (test_low_stock_list) ... ok
test_negative_price_rejected (test_negative_price_rejected) ... ok
test_sell_reduces_stock_and_returns_amount (test_sell_reduces_stock_and_returns_amount) ... ok
test_unknown_item (test_unknown_item) ... ok
----------------------------------------------------------------------
Ran 5 tests in 0.002s
OK
```

Figure 7: Output of the unit tests (six tests, all passing)

```python
import unittest
from store import Store, OutOfStock

class TestStore(unittest.TestCase):
    def setUp(self):
        self.s = Store()
        self.s.add_item("Rice 5kg", 320.0, 40)
        self.s.add_item("Sugar 1kg", 45.0, 8)

    def test_sell_reduces_stock_and_returns_amount(self):
        self.assertEqual(self.s.sell("Rice 5kg", 3), 960.0)
        self.assertEqual(self.s.all_items()[0][3], 37)

    def test_cannot_sell_more_than_stock(self):
        with self.assertRaises(OutOfStock):
            self.s.sell("Sugar 1kg", 9)
```

```
    def test_negative_quantity_rejected(self):
        with self.assertRaises(ValueError):
            self.s.sell("Rice 5kg", -5)

    def test_unknown_item(self):
        with self.assertRaises(KeyError):
            self.s.sell("Tea", 1)

    def test_low_stock_list(self):
        self.assertEqual(self.s.low_stock(10), [("Sugar 1kg", 8)])

    def test_negative_price_rejected(self):
        import sqlite3
        with self.assertRaises(sqlite3.IntegrityError):
            self.s.add_item("Bad", -1, 1)
```

Each test is independent, because setUp creates a fresh in-memory database before every test. Tests were written for the normal case (a successful sale), the limits (selling more than the stock) and wrong input (an unknown item, a negative price, a negative quantity), which is the usual pattern for good tests.

3.9 API basics

An Application Programming Interface (API) allows one program to use the services of another. A web API is used over HTTP: the client sends a request to an endpoint (a URL) with a method such as GET (read), POST (create or act), PUT (change) or DELETE (remove), and the server returns a response with a status code and, usually, JSON data. Common status codes are 200 (OK), 201 (created), 400 (bad request), 404 (not found), 409 (conflict) and 500 (server error). A good API uses clear names, returns suitable codes and error messages, and is documented. The stock application exposes two read endpoints and one action endpoint, and was tested with curl.

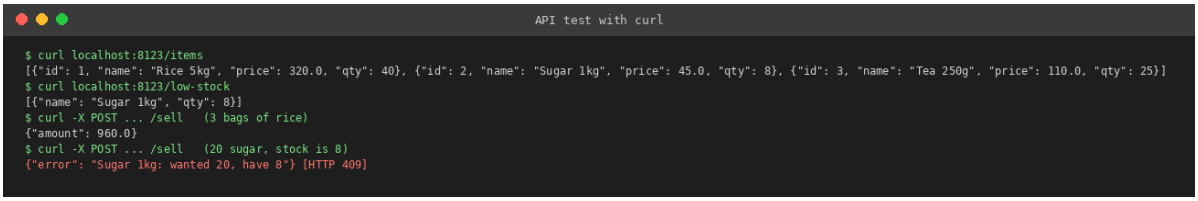


Figure 8: Calling the API with curl

The first call lists the items, the second the low-stock items, the third records a sale of three bags of rice and returns the amount of Rs. 960, and the fourth tries to sell 20 bags of sugar when only 8 are in stock and receives the code 409 with a clear message. The student noted that the API layer only translates between HTTP and the Store class; the rules stay in store.py.

3.10 Deployment basics

Deployment makes the software available to its users. The steps are to prepare a build (collect the code and the dependencies, often listed in a file such as requirements.txt), configure the environment (settings and secrets kept outside the code), choose the hosting (a server, a cloud service or a platform), release the new version, and monitor it. Common practices are to keep separate development, testing and production environments, to release in small steps, to keep a way of returning to the previous version (rollback), to take backups of the database and to keep logs. Continuous integration runs the tests automatically on every change, and continuous delivery releases tested changes with little manual work. Containers such as Docker package an application with everything it needs so that it runs in the same way everywhere. The student prepared a deployment checklist (see section 13) for the stock application, but did not deploy it to a public server, since only dummy data was used.

4. Week-wise Plan and Execution

Week	Days	Activities
Week 1	Days 1-6	Orientation and foundations: company induction, introduction to software development, SDLC overview and requirement analysis, each with a study session and a practice task.
Week 2	Days 7-12	Design and tools: system design basics, version control with Git and coding practices.
Week 3	Days 13-18	Data and quality: database design, debugging techniques and software testing.
Week 4	Days 19-24	Integration: API basics, deployment basics, the Software Application Development project, a case study exercise and a mock practical exercise.
Week 5	Days 25-28	Project closure: supervisor review and feedback, internship report drafting, presentation preparation and final review.

5. Week-wise Narrative

Week 1: The first week was devoted to orientation and foundations. After company induction, the student studied the SDLC and the waterfall and agile models and wrote requirements and user stories for the stock application.

Week 2: In the second week the student drew the architecture and data-flow diagrams, practised Git by creating a repository, making commits and a branch, and learnt coding practices such as naming and documentation.

Week 3: The third week was devoted to database design with normalisation, to debugging, where a real bug was found and traced, and to unit testing.

Week 4: In the fourth week the student built the JSON API and tested it with curl, studied deployment, completed the project, a case study exercise and a mock practical exercise.

Week 5: The last week was devoted to project closure: supervisor review and feedback, drafting of the internship report, preparation of a presentation and final review before submission.

6. Topic-wise Learning in Detail

6.1 SDLC Overview

Concept: Phases of the life cycle and the waterfall and agile models. **Practice carried out:** Drawing the SDLC and agile cycles and comparing the models for three project types.

6.2 Requirement Analysis

Concept: Collecting, documenting and validating requirements. **Practice carried out:** Writing six requirements and a user story with acceptance criteria (section 3.2).

6.3 System Design Basics

Concept: Architecture, modules, data flow and simple design diagrams. **Practice carried out:** Drawing the layered architecture and a data flow diagram (Figure 3).

6.4 Version Control (Git)

Concept: Commits, branches, merges and repositories. **Practice carried out:** Creating a repository, committing in steps, using a branch and merging (Figure 4).

6.5 Coding Practices

Concept: Naming, code structure, comments and clean code. **Practice carried out:** Writing store.py with clear names, documentation strings and error handling.

6.6 Database Design

Concept: Tables, keys, relationships and normalisation. **Practice carried out:** Designing the items table with constraints and an ER diagram of the shop (Figure 5).

6.7 Debugging Techniques

Concept: Finding and fixing bugs using logs and breakpoints. **Practice carried out:** Reproducing and fixing the negative quantity bug (Figure 6).

6.8 Software Testing

Concept: Test cases, unit testing and bug reporting. **Practice carried out:** Writing six unit tests and a bug report (Figure 7).

6.9 API Basics

Concept: Requests, responses, endpoints and using simple APIs. **Practice carried out:** Building two GET endpoints and one POST endpoint and testing them with curl (Figure 8).

6.10 Deployment Basics

Concept: Preparing a build, hosting and releasing an application. **Practice carried out:** Preparing a deployment checklist with the build, configuration, release and rollback steps.

7. Professional Ethics and Good Practice in Software - Points Studied

Software affects the money, privacy and safety of people, so professional responsibility is part of the work. The points studied are summarised below.

- **Quality and honesty:** do not claim that software is tested when it is not; report known bugs honestly.
- **Security:** validate every input, use parameterised queries, protect passwords and never put secrets in the code.
- **Privacy:** collect only the data that is needed and protect it; the Digital Personal Data Protection Act, 2023 applies.
- **Licences:** follow the licence of any library or code used, and give credit.

- **Originality:** do not copy another person's code without permission; academic and professional honesty apply.
- **Accessibility:** build software that people with different abilities and devices can use.
- **Responsibility for impact:** think about who may be harmed by a mistake in the software and test accordingly.

8. Process Flows Studied

8.1 Steps of a feature, from request to release

- Write the requirement as a user story with acceptance criteria.
- Create a branch for the feature.
- Write the tests, then the code, and run all tests.
- Commit in small steps with clear messages and ask for a review.
- Merge into main, deploy in a small release and watch for problems.

8.2 Steps of finding a bug

- Reproduce the bug with the smallest example.
- Find where the wrong value first appears, using prints, logs or a debugger.
- Understand the cause, not only the symptom.
- Fix the cause on a branch and add a test that fails before and passes after the fix.
- Run all tests, merge and note the fix in the log.

9. Worked Examples with Sample (Dummy) Data

Worked example 1 - writing a test first. Requirement R3 says that a sale larger than the stock must be refused. The test was written first, and it failed because the function did not yet raise an error. The code was then written until the test passed. Writing the test first makes the requirement concrete and ensures that the feature has a test.

```
def test_cannot_sell_more_than_stock(self):
    with self.assertRaises(OutOfStock):
        self.s.sell("Sugar 1kg", 9)     # only 8 in stock
```

Worked example 2 - estimating effort. The stock application has five features (add item, sell, low stock, API, tests). The student estimated one day for each of the first four and one day for tests, and a further 20% for review and corrections, giving 5 x 1.2 = 6 working days. The real work took 6.5 days. An estimate is never exact, but comparing it with the actual time improves the next one.

Worked example 3 - the cost of a late bug. A study often quoted in software courses says that a defect found in production can cost many times more to fix than one found at the requirement stage. A simple illustration from the practice: the negative-quantity bug was fixed in ten minutes, because it was found in testing; had it reached the shop, wrong stock figures would have had to be corrected by hand for every affected item.

10. Project: Software Application Development Project

Problem statement. The owner of a fictitious shop in Gaya keeps the stock in a register and often discovers too late that an item has run out, and sometimes sells more than the stock. The owner wants a simple program to record the items, record a sale, refuse an impossible sale and show the items that are running low. All data is dummy data.

10.1 Method

- Write the requirements and user stories (Table in section 3.2).
- Design the layers and the items table (Figures 3 and 5).
- Build store.py and test it with unit tests, committing in small steps.
- Find and fix a bug through a feature branch and merge it.
- Expose the application through a JSON API and test it with curl.

10.2 Result summary

Item	Result
Code	store.py (about 40 lines) and app.py (about 40 lines), standard library only
Tests	6 unit tests, all passing
Git history	4 commits, one feature branch and one merge commit
API	GET /items, GET /low-stock, POST /sell with codes 200, 404 and 409
Bug found and fixed	Negative sale quantity increased the stock; now rejected with an error
Stock value (dummy data)	Rs. 15,910

10.3 Possible improvements

- Add the customers and sales tables of Figure 5 and a sales history.
- Add a PUT endpoint to restock an item and a DELETE endpoint to remove one.
- Add a login and role (owner and staff) to the API.
- Add automatic reorder suggestions based on the sales of the past month.

- Run the tests automatically on every push using a continuous integration service.

10.4 Limitations

The application uses the standard library only, has no login, stores data in memory when used in the tests, and was not deployed to a public server. It is a learning project, and a real shop system would need security, backups, a user interface and much more testing. The student noted that the learning lies in following the process from requirements to a tested release.

11. Case Study Exercise: Fixing a Slow Report in a School Management System

A fictitious school management system in Patna takes about 40 seconds to produce the monthly fee-due report for 1,200 students. The head clerk complains that the system 'hangs'. The student was asked to find the cause with the mentor.

Investigation. The problem was reproduced with the data of 1,200 students. The logs showed that the program ran one query to get the list of students and then one more query for each student to find the fees paid, which makes 1,201 queries. Each query took about 30 milliseconds, so 1,201 x 30 ms = about 36 seconds, matching the delay. This is known as the N+1 query problem.

Fix. The queries were replaced by a single query that joins the students and fees tables and adds the amounts with GROUP BY. The report then took about 0.2 seconds. A test was added that compares the output of the old and the new method for 20 students to be sure that the report is unchanged, and an index on the student_id column of the fees table was added.

Lessons. Measure before changing; find the cause and not only the symptom; keep a test that proves the result is the same; and think about the size of the data from the beginning. The figures in the case are illustrative.

12. Mock Practical Exercise: Building and Releasing a Small Feature

A mock practical exercise was conducted at the end of the learning phase to check that the student could carry out a complete small development cycle independently within a fixed time. The task was to add a 'sales total for the day' feature to the stock application. The steps followed were:

1. Requirement: 'As an owner I want the total of today's sales' was written with an acceptance criterion.
2. Design: a sales table with item, quantity, amount and date was designed.
3. Branch: a feature branch was created in Git.
4. Tests: tests for the total of zero, one and many sales were written first.
5. Code: the function was written until the tests passed and the earlier tests still passed.
6. API: a GET /sales-total endpoint was added and checked with curl.
7. Review: the code was read by the supervisor and two naming points were corrected.
8. Merge: the branch was merged with a clear message and the change was noted in the log.

13. Sample Checklists Prepared

Before-commit checklist

- All tests pass and new code has tests.
- Names are clear and there is no unused code.
- No passwords or keys are in the code.
- The commit message says what and why.

Deployment checklist

- Dependencies listed; configuration kept outside the code.

- Database backed up; migration tested on a copy.
- Rollback plan ready; logs and monitoring in place.
- Release notes written.

14. Tools and Resources

The following tools and resources were used during the internship: Python and its standard library (sqlite3, unittest, http.server), SQLite, Git, a code editor, curl for testing the API, a drawing tool for the diagrams, MS Word and PowerPoint for the report and presentation, case studies and templates provided by the agency, and regular mentor feedback.

15. Programme at a Glance

The programme was in the domain of Software Development with RKNEXORA PRIVATE LIMITED as the agency. It required a minimum of 120 hours over 28 learning days, organised in three phases: Orientation (Days 1-2), Learning and Practice (Days 3-22) and Project and Review (Days 23-28). Attendance was recorded in the Internship Attendance Log, signed by the supervisor (see Attendance Sheet).

The approximate distribution of the effort across the topics is shown below.

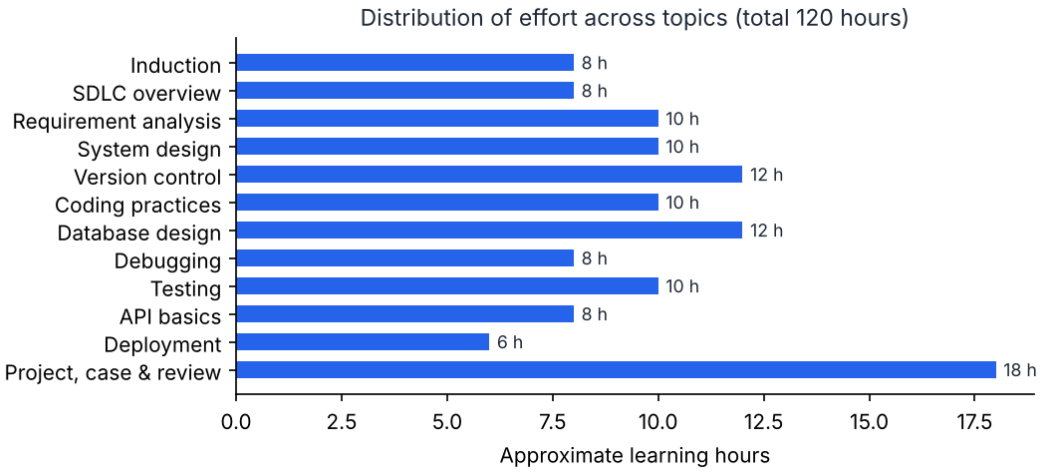


Figure 9: Distribution of learning effort across topics (120 hours)

LEARNING OUTCOME

a. Knowledge Acquired

1. The phases of the SDLC and the waterfall and agile models.
2. Writing functional and non-functional requirements and user stories.
3. Layered architecture and simple design diagrams.
4. Git commits, branches and merges.
5. Clean coding practices and code review.
6. Tables, keys, constraints and normalisation.
7. Systematic debugging.
8. Unit testing and bug reporting.
9. Web APIs, methods and status codes.
10. Build, release, rollback and continuous integration.

Glossary of Terms Learned

- **SDLC** - Software Development Life Cycle.
- **Waterfall / Agile** - A step-by-step model / a short-cycle model.
- **Sprint** - A short, fixed period of agile work.
- **Requirement** - A statement of what the software must do.
- **User story** - A requirement written from the user's side.
- **Repository** - A project folder with its history.
- **Commit / Branch / Merge** - A saved snapshot / a separate line of work / joining branches.
- **Refactoring** - Improving code without changing its behaviour.
- **Primary key / Foreign key** - Identifies a row / links to another table.
- **Normalisation** - Removing repeated data in tables.
- **Bug** - A flaw in software.
- **Unit test** - A test of one small part of the code.
- **Regression** - A feature that worked and then breaks.
- **API** - Application Programming Interface.
- **Endpoint** - A URL of an API.
- **JSON** - A text format for data.
- **SQL injection** - An attack through input that contains SQL.
- **CI/CD** - Continuous integration and delivery.
- **Rollback** - Returning to a previous version.

b. Skills Learned

Technical skills. Writing requirements; drawing architecture and ER diagrams; using Git; writing clean Python; designing SQL tables; debugging; writing unit tests; building and testing a JSON API; and preparing a deployment checklist.

Professional skills. Logical and analytical thinking; attention to detail; patience in debugging; teamwork and willingness to accept code review; clear written communication; time management; and a habit of continuous learning.

These skills were developed through repeated practice and daily documentation. Finding and fixing a bug with a test built patience and confidence, working in small commits built discipline, and explaining the design to the supervisor built the ability to express technical ideas in plain words.

c. Observed Attitude and Gained Values

1. **Quality mindset:** a feature is complete only when it is tested.
2. **Honesty:** report bugs and limits openly instead of hiding them.
3. **Care for users and their data:** validation, privacy and security are part of the job.
4. **Learning from feedback:** code review is a chance to improve.
5. **Discipline and punctuality:** regular reporting at fixed timings and daily attendance records.

d. Challenges Faced

Understanding Git branches and merges was handled by drawing the history and practising on a dummy repository.

Finding the cause of a bug, not just the place, was handled by reproducing it with the smallest example and printing the values.

Choosing what to test was handled by writing tests for normal, limit and wrong inputs.

Designing tables without repeated data was handled by applying the normal forms step by step.

Understanding HTTP status codes was handled by calling the API and reading the real responses.

Managing time between coding, documentation and report writing was handled by dividing work week by week and using review days for corrections.

e. Career Pathways

The knowledge gained supports careers such as Junior Software Developer, Software Tester or QA Trainee, Web or Backend Developer (junior), Technical Support Engineer, Database Assistant, Business Analyst (technical) and DevOps Trainee. Opportunities exist in IT services, product companies, start-ups, banks, e-commerce, ed-tech, health-tech and government digital projects. Related higher-study options include BCA, B.Tech, MCA, M.Sc. in computer science, and certificate courses in Python, Java, SQL, Git, testing and cloud computing. A good next step is to build two or three small projects, host them on GitHub, learn a web framework and practise data structures and algorithms.

CONCLUSION

The internship in Software Development was a valuable learning experience that integrated academic knowledge with practical activity. The programme achieved its objectives through structured learning, guided practice, assignments, supervisor review and documentation.

The student gained a clear understanding of how software is planned, designed, built, tested and released - through the SDLC, requirement analysis, design, Git, clean coding, database design, debugging, testing, APIs and deployment - and acquired practical experience by developing a tested stock management application with a JSON API. The experience has strengthened the student's employability and provides a foundation for further study and work in software development.

Reflection

Looking back, the most useful lesson of the internship was that software development is a process of small, checked steps. A bug that would have been invisible in manual use was caught by a test, and the fix could be merged safely because the earlier tests still passed. Writing the requirements first made the later coding easier, and using Git removed the fear of changing working code. The habits of testing, clear naming and honest reporting apply to every kind of software work.

Objectives and Achievement

The objective of the SDLC and requirements was achieved through the life-cycle diagrams and the requirement table. The objective of design and database was achieved through the architecture and ER diagrams. The objective of Git and coding was achieved through the repository history and clean code. The objective of debugging and testing was achieved through the bug fix and six unit tests. The objective of APIs and deployment was achieved through the tested API and the deployment checklist.

Future Scope

The knowledge gained can support further study of computer science and careers in software, testing and data. With the growth of digital services, start-ups and government digital platforms in Bihar and across India, there is a continuing need for developers who write clean, tested and secure code.

Suggestions for Improvement of the Programme

1. Provide access to a shared Git service for group projects with code review.
2. Include a short module on a web framework along with the standard library.
3. Add a module on automated testing and continuous integration.
4. Continue regular mentor feedback and progress reviews.
5. Encourage students to maintain detailed daily documentation.

References

1. Kolb, D. A. (1984). Experiential Learning: Experience as the Source of Learning and Development. Prentice Hall.
2. University Grants Commission (UGC). Internship guidelines for higher education institutions.
3. Ministry of Education, Government of India. National Education Policy 2020.
4. Sommerville, I. Software Engineering. Pearson.
5. Pressman, R. S. Software Engineering: A Practitioner's Approach. McGraw Hill.
6. Martin, R. C. Clean Code. Prentice Hall.
7. Chacon, S. and Straub, B. Pro Git. Apress (free at git-scm.com).

8. Python Software Foundation. Python documentation: unittest, sqlite3 and http.server.
9. Elmasri, R. and Navathe, S. Fundamentals of Database Systems. Pearson.
10. RK NEXORA internship programme handbook.

ATTENDANCE SHEET

The Internship Attendance Log issued by the IPO, with the supervisor's signature for each day, is to be attached on this page.

[Attach the signed Internship Attendance Log here]